

# Reviewer Pack — Minimal Order of p-Adic p-Group Representations and Communic...

Entry 348b9b773a85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 13:21:46 UTC

## Minimal Order of p-Adic p-Group Representations and Communication Complexity Rank Correlation

Entry ID: 348b9b773a85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 13:21:46 UTC

### 1. Conjecture

**Field A** (mathematical branch): p-adic Group Theory **Field B** (complexity object): Communication Complexity

#### Statement:

The minimal order of a p-group representation associated with an n-bit communication complexity problem is bounded above by the square root of the number of distinct possible communication channels, i.e.,  $O(\sqrt{n})$ .

#### Rationale (proposer's reasoning):

p-Group representations encode combinatorial information that could be relevant for distributing tasks in a distributed system. The minimal order measures the complexity of these representations and might provide insights into the intrinsic difficulty of solving communication complexity problems.

**Taxonomy category:** p-adic\_group\_theory\_communication\_complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** d49e7d0020cadede

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

A conjecture is supported if the mean minimal order of p-group representations across all n-bit communication complexity instances, computed from 30 random seeds, is less than or equal to the square root of n, i.e.,  $\text{mean}(\text{minimal\_order}) \leq \sqrt{n}$ . The conjecture is falsified if any seed produces a minimal order greater than  $\sqrt{n}$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.90       | SAFE             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** ADJACENT\_OK against 8 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - p-adic group theory AND communication complexity AND representation  
- minimal order p-group representations AND communication complexity rank - p-adic p-group representations AND  $O(\sqrt{n})$  communication channels

**Top relevant hits considered:** - [http://arxiv.org/abs/1001.0462v2] Representation Theory of Finite Groups - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for  $P$ -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/1701.07658v2] On unitarity of some representations of classical p-adic groups I - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/1505.00626v3] Minimal dimension of faithful representations for p-groups - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1701.07662v2] On unitarity of some representations of classical p-adic groups II - [http://arxiv.org/abs/hep-th/9506097v2] p-Adic TGD: Mathematical Ideas

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_complexity_instance(n):
        return [random.randint(0, 1) for _ in range(n)]

    def compute_p_group_representation(cc_instance):
        # Placeholder function to simulate computing a p-group representation
        # This is a dummy implementation and should be replaced with actual logic
        return len(set(cc_instance))

    def correlation(x, y):
        mean_x = sum(x) / len(x)
        mean_y = sum(y) / len(y)
        cov = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / len(x)
        var_x = sum((xi - mean_x) ** 2 for xi in x) / len(x)
        return cov / math.sqrt(var_x * sum((yi - mean_y) ** 2 for yi in y))

    n_values = [5, 10, 15, 20, 30, 40]
    minimal_orders = []
    cc_ranks = []
```

```

for n in n_values:
    for _ in range(5): # Sample 5 instances per size
        cc_instance = generate_communication_complexity_instance(n)
        minimal_order = compute_p_group_representation(cc_instance)
        minimal_orders.append(minimal_order)
        cc_ranks.append(sum(cc_instance))

if len(minimal_orders) < 30:
    return {
        "metric_name": "minimal_order",
        "metric_value": None,
        "instances_tested": len(minimal_orders),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mean_minimal_order = sum(minimal_orders) / len(minimal_orders)
cc_rank_correlation = correlation(cc_ranks, minimal_orders)

return {
    "metric_name": "minimal_order",
    "metric_value": mean_minimal_order,
    "instances_tested": 30,
    "n_max": max(n_values),
    "conjecture_holds": mean_minimal_order <= math.sqrt(max(n_values)),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n_max={result['n_max']}, cc_instance={result['cc_instance']}")
    else:
        print("RESULT: INCONCLUSIVE not_enough_support")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TRIAL: {'metric\_name': 'minimal\_order', 'metric\_value': 1.9666666666666666, 'instances\_tested': 30,

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7d44bcf.py", line 74, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7d44bcf.py", line 58, in run_trial
    cc_rank_correlation = correlation(cc_ranks, minimal_orders)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7d44bcf.py", line 34, in correlation
    return cov / math.sqrt(var_x * sum((yi - mean_y) ** 2 for yi in y))
           ~~~~^~~~~~
ZeroDivisionError: float division by zero
```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means we cannot verify if the mean minimal order of p-group representations across all n-bit communication complexity instances is less than or equal to the square root of n. | next: Re-run the test without errors to confirm the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13185        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 16927        |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 17737        |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 13592        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11267        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14517        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9693         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15753        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8970         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 121643 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/348b9b773a85.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/348b9b773a85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/348b9b773a85.tar.gz` (if generated)